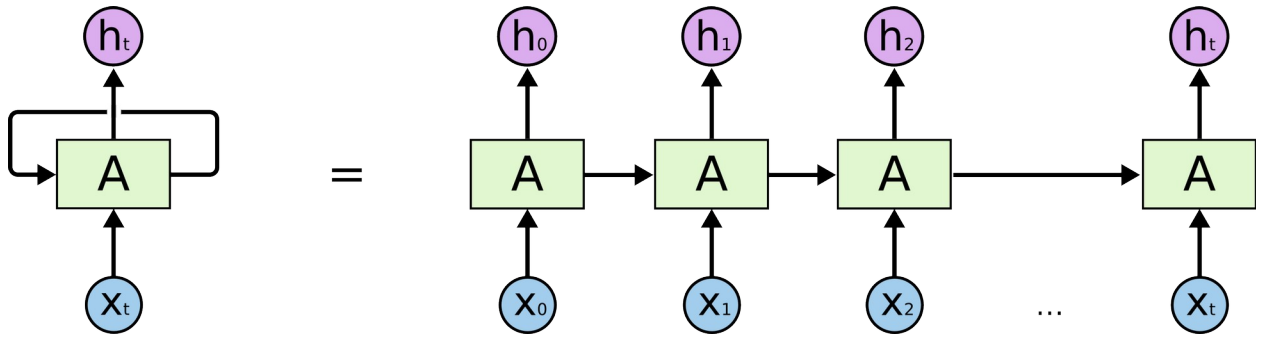
Natural Language Processing (Sentiment Analysis) with Recurrent Neural Networks

Word Embeddings

Word embeddings keeps the order of words intact and encodes similar words with similar labels. It attempts to encode the frequency and order of words as well as the meaning of those words in the sentence. It encodes each word as a dense vector that represents its context in the sentence.

Word embeddings are learned by looking at many different training examples. An *embedding layer* can be added to the beginning of the model and be trained for correct embeddings for words. We can also use pre-trained embedding layers.

##Recurrent Neural Networks An RNN will process one word at a time while maintaining an internal memory of what it has already seen. This allows it to treat words differently based on their order in a sentence and to slowly build an understanding of the entire input, one word at a time. The text data are treated as a sequence to pass one word at a time to the RNN.



Source: https://colah.github.io/posts/2015-08-Understanding-LSTMs/

where **ht** is output at time t, **xt** is input at time t, and **A** is Recurrent Layer (loop).

This is a **simple RNN layer**. The recurrent layer processes words or input one at a time in a combination with the output from the previous iteration. As we progress further in the input sequence, we build a better understanding of the text as a whole.

LSTM

A Long Short-Term Memory (LSTM) RNN works similarly but adds a way to access inputs from any timestep in the past. With LSTM, we have a long-term memory data structure storing all the previously seen inputs as well as when we saw them. This adds to the complexity of our network and allows it to discover more useful relationships between inputs and when they appear.

Sentiment Analysis

Sentiment Analysis (from Wikipedia):

the process of computationally identifying and categorizing opinions expressed in a piece of text, especially in order to determine whether the writer's attitude towards a particular topic, product, etc. is positive or negative.

In this example, we will classify movie reviews as potitive, negative, or neutral.

This guide is based on the following tensorflow tutorial:
https://www.tensorflow.org/tutorials/text/text_classification_rnn

Movie Review Dataset

Load in the IMDB movie review dataset from Keras.

This dataset contains 25,000 reviews from IMDB where each one is already preprocessed and has a label as either positive or negative. Each review is encoded by integers that represents how common a word is in the entire dataset. For example, a word encoded by the integer 3 means that it is the 3rd most common word in the dataset.

```
# %tensorflow_version 2.x
from keras.datasets import imdb
from keras.preprocessing import sequence
import keras
import tensorflow as tf
import os
import numpy as np

VOCAB_SIZE = 88584

MAXLEN = 250
BATCH_SIZE = 64

(train_data, train_labels), (test_data, test_labels) =
imdb.load_data(num_words = VOCAB_SIZE)

display(len(train_data))
display(len(train_data[0]))
display(len(test_data))
```

```
25000
```

```
218
```

```
25000
```

```
# Lets look at one review
display(len(train_data[0]))
# display(train_data[0])
```

```
218
```

More Preprocessing

The reviews are of different lengths. We cannot pass different length data into our neural network. Therefore, we must make each review the same length. To do this, we will follow the procedure below:

- if the review is greater than 250 words, trim off the extra words
- if the review is less than 250 words, add the necessary amount of 0's to make it equal to 250.

```
from tensorflow.keras.preprocessing.sequence import pad_sequences

train_data = pad_sequences(train_data, MAXLEN)
test_data = pad_sequences(test_data, MAXLEN)

# train_data = sequence.pad_sequences(train_data, MAXLEN)
# test_data = sequence.pad_sequences(test_data, MAXLEN)
```

We load the encodings from the dataset and use them to encode the review data.

```
# Build an encode function
word_index = imdb.get_word_index()

word_index = {k: (v + 3) for k, v in word_index.items()}

word_index["<PAD>"] = 0
word_index["<START>"] = 1
word_index["<UNK>"] = 2
word_index["<UNUSED>"] = 3

reverse_word_index = {value: key for (key, value) in
word_index.items()}

def encode_text(text):
  tokens = tf.keras.preprocessing.text.text_to_word_sequence(text)
  tokens = [word_index.get(word, 2) for word in tokens]
  return pad_sequences([tokens], MAXLEN)[0]

text = "that movie was just amazing, so amazing"
encoded = encode_text(text)
print(encoded)
```

```
[  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
0
   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
0
   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
0
   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
0
   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
```

```
0
   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
0
   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
0
   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
0
   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
0
   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
0
   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
0
   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
0
   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
0
   0   0   0   0   0   0   0   0   0  15  20  16  43 480  38 480]
```

```
# Build a decode function

reverse_word_index = {value: key for (key, value) in
word_index.items()}

def decode_integers(integers):
  text = ""
  for num in integers:
    if num != 0: # Ignore padding
      text += reverse_word_index.get(num, "<UNK>") + " "
  return text.strip()

  # .strip() is a cleaner way to remove the trailing space than
text[:-1]
  return text.strip()

print("Your encoded text:", decode_integers(encoded))
print("First training review:", decode_integers(train_data[0]))
display(train_labels[0])
```

```
Your encoded text: that movie was just amazing so amazing
First training review: <START> this film was just brilliant casting
location scenery story direction everyone's really suited the part
they played and you could just imagine being there robert redford's is
an amazing actor and now the same being director norman's father came
from the same scottish island as myself so i loved the fact there was
a real connection with this film the witty remarks throughout the film
were great it was just brilliant so much that i bought the film as
soon as it was released for retail and would recommend it to everyone
to watch and the fly fishing was amazing really cried at the end it
was so sad and you know what they say if you cry at a film it must
```

```
have been good and this definitely was also congratulations to the two little boy's that played the part's of norman and paul they were just brilliant children are often left out of the praising list i think because the stars that play them all grown up are such a big profile for the whole film but these children are amazing and should be praised for what they have done don't you think the whole story was so lovely because it was true and was someone's life after all that was shared with us all

np.int64(1)
```

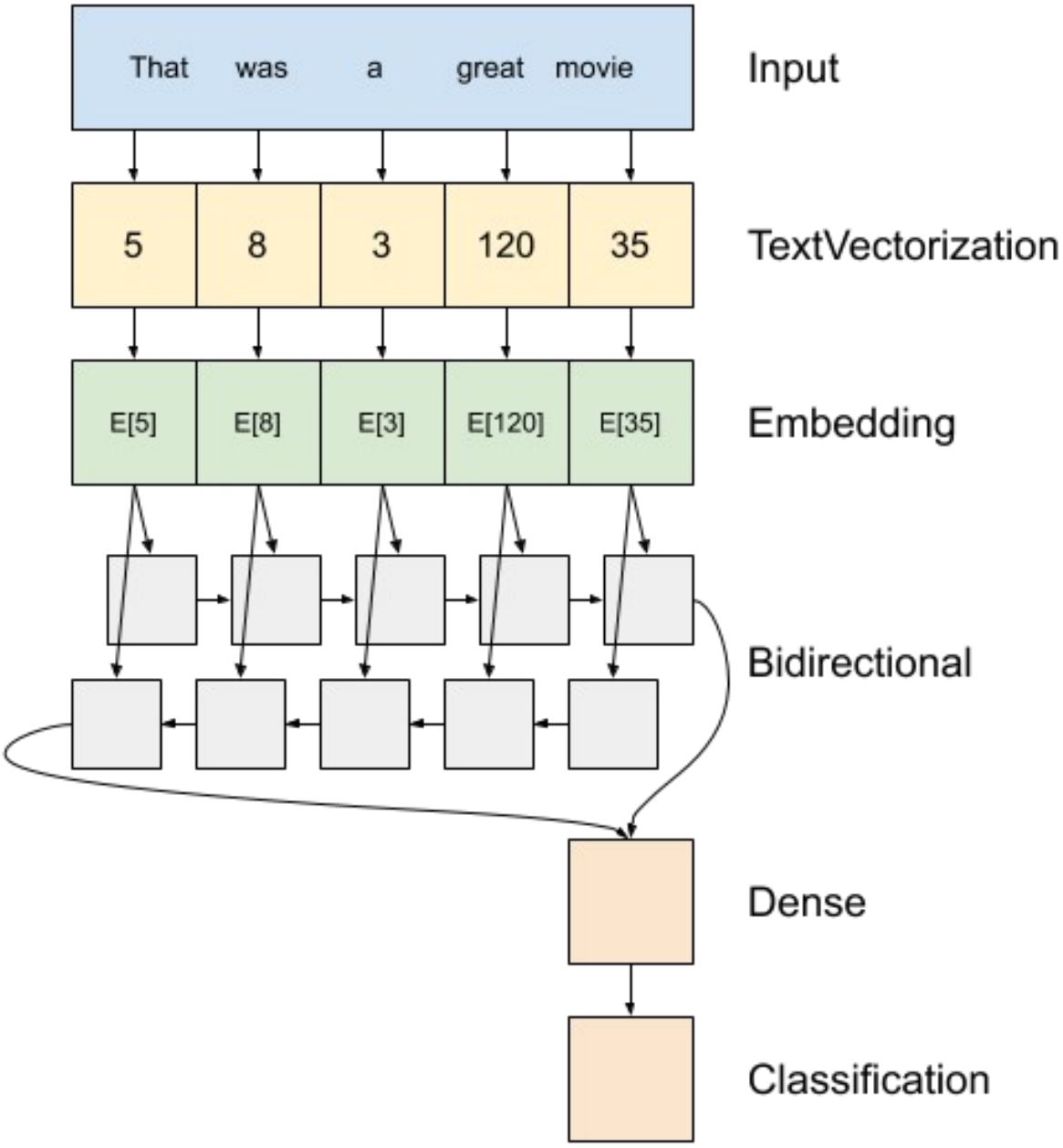
Create the Model

An **embedding layer** stores one vector per word. When called, it converts the sequences of word indices to sequences of vectors. These vectors are trainable. After training (on enough data), words with similar meanings often have similar vectors.

We use a word embedding layer as the first layer. Then we add an LSTM layer to the model followed by a dense layer that outputs a node of the predicted sentiment.

In the following, 32 stands for the output dimension of the vectors generated by the embedding layer.

The following picture is for reference. We don't have TextVectorization layer here. We don't use Bidirectional RNN here.




```
model = tf.keras.Sequential([
    tf.keras.layers.Embedding(VOCAB_SIZE, 32),
    tf.keras.layers.LSTM(32),
    tf.keras.layers.Dense(1, activation="sigmoid")
])

# model = tf.keras.Sequential([
#     tf.keras.layers.Embedding(
#         input_dim=VOCAB_SIZE,
#         output_dim=32,
```

```
#         # Use masking to handle the variable sequence lengths
#         mask_zero=True),
#     tf.keras.layers.Bidirectional(tf.keras.layers.LSTM(32)),
#     tf.keras.layers.Dense(32, activation='relu'),
#     tf.keras.layers.Dense(1, activation='sigmoid')
# ])

model.summary()
```

```
Model: "sequential_1"
```

```
┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━┓
┃ Layer (type)                         ┃ Output Shape                ┃         Param # ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━┩
│ embedding_1 (Embedding)              │ ?                           │   0 (unbuilt) │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────┤
│ lstm_1 (LSTM)                        │ ?                           │   0 (unbuilt) │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────┤
│ dense_1 (Dense)                      │ ?                           │   0 (unbuilt) │
└──────────────────────────────────────┴─────────────────────────────┴─────────────────┘
```

```
 Total params: 0 (0.00 B)

 Trainable params: 0 (0.00 B)

 Non-trainable params: 0 (0.00 B)
```

###Training Compile and train the model.

```
model.compile(loss="binary_crossentropy", optimizer="rmsprop",
metrics=['acc'])
#
model.compile(loss=tf.keras.losses.BinaryCrossentropy(from_logits=True
),
#               optimizer=tf.keras.optimizers.Adam(1e-4),
#               metrics=['accuracy'])
history = model.fit(train_data, train_labels, epochs=2,
validation_split=0.2)
```

```
Epoch 1/2
625/625 ━━━━━━━━━━━━━━━━━━━━ 32s 48ms/step - acc: 0.7688 - loss:
0.4723 - val_acc: 0.8658 - val_loss: 0.3209
```

```
Epoch 2/2
625/625 ━━━━━━━━━━━━━━━━━━━━ 30s 48ms/step - acc: 0.8874 - loss:
0.2870 - val_acc: 0.8846 - val_loss: 0.2987
```

Evaluate the model on the test data to see how well it performs.

```
results = model.evaluate(test_data, test_labels)
print(results)
```

```
782/782 ━━━━━━━━━━━━━━━━━━━━ 12s 16ms/step - acc: 0.8801 - loss:
0.3018
[0.3017704486846924, 0.880079984664917]
```

Make Predictions

Use the trained network to make predictions on our own reviews.

We need to convert our review into the form that the network can understand. Call the function encode_text().

```
# Make a prediction

def predict(text):
  encoded_text = encode_text(text)
  pred = np.array([encoded_text])
  result = model.predict(pred)
  print(result[0])

positive_review = "That movie was! really loved it and would great
watch it again because it was amazingly great"
predict(positive_review)

negative_review = "that movie really sucked. I hated it and wouldn't
watch it again. Was one of the worst things I've ever watched"
predict(negative_review)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 160ms/step
[0.9795503]
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 38ms/step
[0.06183567]
```

Sources

1. Chollet François. Deep Learning with Python. Manning Publications Co., 2018.
2. “Text Classification with an RNN ： TensorFlow Core.” TensorFlow, www.tensorflow.org/tutorials/text/text_classification_rnn.
3. “Understanding LSTM Networks.” Understanding LSTM Networks -- Colah's Blog, https://colah.github.io/posts/2015-08-Understanding-LSTMs/.